MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

VREMERE ADRIAN, FAF-232

Report

Laboratory Work n.2.2

Button Press Duration Monitoring with FreeRTOS Preemptive Scheduler

on Embedded Systems

Checked by:
Martîniuc Alexei, *univ. asist.*
FCIM, UTM

Chişinău – 2026

1. Domain Analysis

1.1. Objective of the Laboratory Work

The objective of this laboratory work is to port the bare-metal button press monitoring application developed in Lab 2.1 to a preemptive real-time operating system (FreeRTOS) running on the Arduino Mega 2560 microcontroller. The system retains the same functional requirements — monitoring button press durations, providing visual LED feedback, and periodically reporting statistics over the STDIO serial interface — but replaces the cooperative non-preemptive scheduler with FreeRTOS preemptive task management.

The key learning objectives include understanding preemptive scheduling in embedded real-time operating systems, using binary semaphores as lightweight event signaling mechanisms between producer and consumer tasks, employing mutexes with priority inheritance to protect shared data from concurrent access, and comparing the design trade-offs between bare-metal cooperative and RTOS-based preemptive approaches.

1.2. Problem Definition

The laboratory requires the implementation of the same button press monitoring system as Lab 2.1, restructured to use FreeRTOS preemptive scheduling with explicit synchronization. The functional requirements are identical:

1. Task 1 — Button Detection and Duration Measurement: monitor the push button, debounce press/release transitions using a finite state machine, measure each press duration in milliseconds, and visually signal a short press (< 500 ms) with a green LED or a long press (≥ 500 ms) with a red LED.

2. Task 2 — Statistics Accumulation and Yellow LED Blink: on each detected press, increment press counters and duration accumulators, then drive a rapid yellow LED blink sequence: 5 blinks for a short press and 10 blinks for a long press.

3. Task 3 — Periodic STDIO Reporting: every 10 seconds, transmit a formatted summary (total presses, short/long counts, average duration) over the serial STDIO interface, then reset the statistics for the next measurement window.

The additional requirements specific to Lab 2.2 are:

1. Use FreeRTOS `xTaskCreate()` to spawn three preemptive tasks with distinct priorities.

2. Use `vTaskDelay()` and `vTaskDelayUntil()` for periodic execution and timing.

3. Use a binary semaphore for event signaling between Task 1 (producer) and Task 2 (consumer).

4. Use a mutex to protect all shared variables from concurrent access, demonstrating priority inheritance.

5. Port the application code with minimal changes, preserving the button FSM, LED management, and reporting logic.

1.3. Used Technologies

FreeRTOS — Preemptive Real-Time Operating System

FreeRTOS is a widely adopted open-source real-time operating system kernel designed for microcontrollers and resource-constrained embedded devices. Unlike the bare-metal cooperative scheduler used in Lab 2.1, FreeRTOS implements preemptive scheduling: the kernel runs a periodic tick interrupt (driven by a hardware timer), and at each tick it evaluates whether a higher-priority task has become ready. If so, the currently running task is immediately suspended (preempted) and the higher-priority task resumes execution. This guarantees that the highest-priority ready task always runs, providing bounded worst-case response latency.

Each FreeRTOS task has its own private stack and a Task Control Block (TCB) that stores the saved CPU context (registers, program counter, stack pointer). Context switching — saving one task's state and restoring another's — is performed transparently by the kernel's PendSV interrupt handler. On the ATmega2560 at 16 MHz, a context switch takes approximately 10–20 μs, which is negligible compared to the application's 10 ms minimum task period.

FreeRTOS provides several delay mechanisms: `vTaskDelay()` suspends the calling task for a specified number of ticks relative to the call time, while `vTaskDelayUntil()` maintains an absolute wake-up time that advances by the period on each call. The latter prevents cumulative drift and is used for Task 3's strict 10-second reporting interval.

Binary Semaphores — Event Signaling

A binary semaphore is a FreeRTOS synchronization primitive that operates as a lightweight event flag. It can be in one of two states: *given* (available) or *taken* (empty). A producer task calls `xSemaphoreGive()` to set the semaphore, and a consumer task calls `xSemaphoreTake()` to wait for it. If the semaphore is empty when `xSemaphoreTake()` is called, the consumer task is moved to the Blocked state with zero CPU overhead until the producer gives the semaphore.

In this application, Task 1 (button measurement) gives the semaphore when a complete press cycle is detected, and Task 2 (statistics) takes the semaphore to wake up and process the event. This replaces the polling approach used in Lab 2.1, where Task 2 checked a volatile boolean flag on every 50 ms tick. The semaphore approach is more efficient: Task 2 consumes zero CPU while waiting, and the wake-up latency is bounded by one RTOS tick plus context switch time.

Mutexes — Shared Resource Protection

A mutex (mutual exclusion) is a FreeRTOS synchronization primitive designed to protect shared resources from concurrent access by multiple tasks. Unlike a binary semaphore, a mutex includes a *priority inheritance* mechanism: if a low-priority task holds the mutex and a high-priority task attempts to acquire it, the kernel temporarily raises the holder's priority to match the waiter's priority. This prevents *priority inversion*, a pathological scenario where a medium-priority task preempts the mutex holder, indirectly blocking the high-priority waiter indefinitely.

In this application, `xSharedDataMutex` protects two shared structures: `g_pressInfo` (written by Task 1, read by Task 2) and `g_stats` (updated by Task 2, read and reset by Task 3). Every access to these structures is bracketed by `xSemaphoreTake()` and `xSemaphoreGive()` on the mutex, ensuring atomicity even under preemptive scheduling.

GPIO and Button Debouncing

The GPIO and button debouncing techniques are identical to those described in Lab 2.1. Pin 7 is configured as `INPUT_PULLUP` for the button (active-LOW), and pins 8, 9, and 10 drive the green, red, and yellow LEDs respectively through 220 Ω current-limiting resistors. The four-state button FSM (Idle, Debounce-Down, Pressed, Debounce-Up) with a 50 ms confirmation window is reused without modification.

UART Serial Communication and STDIO

The STDIO redirection to UART0 via the `StdioSerial` library is reused from Lab 1.1 without modification. Since only Task 3 produces serial output (the periodic report), and the `setup()` banner completes before the scheduler starts, there is no concurrent access to the serial port and no additional synchronization is required for STDIO.

1.4. Hardware Components

The hardware configuration is identical to Lab 2.1 — the same Arduino Mega 2560 board, one push button, and three LEDs with identical pin mapping. Since FreeRTOS is a software-only change (a library linked into the firmware), no physical hardware modifications are required.

Arduino Mega 2560

The Arduino Mega 2560 is a microcontroller development board based on the ATmega2560, an 8-bit AVR microcontroller running at 16 MHz. It provides 54 digital I/O pins, 16 analogue inputs, 256 KB Flash, 8 KB SRAM, and 4096 bytes of EEPROM. The FreeRTOS kernel and three task stacks fit comfortably within the 8 KB SRAM: the compiled firmware uses approximately 944 bytes of RAM (11.5 %).

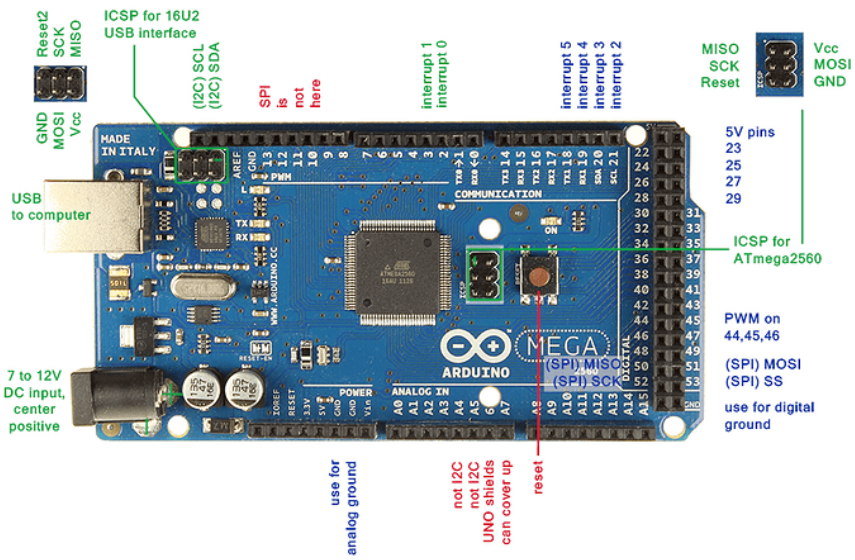

Figure 1.1 *– Arduino Mega 2560 development board*

Push Button

The circuit uses a standard momentary-action tactile push button. When pressed, it connects pin 7 of the microcontroller to GND, producing a LOW reading. The ATmega2560 internal pull-up is enabled, so no external resistor is required. Button bouncing is handled in software by the four-state FSM.

Figure 1.2 *– Momentary tactile push button*

LEDs and Current-Limiting Resistors

Three 5 mm LEDs — green, red, and yellow — provide the visual output. Each is driven through a 220 Ω resistor, resulting in approximately 13.6 mA forward current, within the ATmega2560 GPIO specifications.

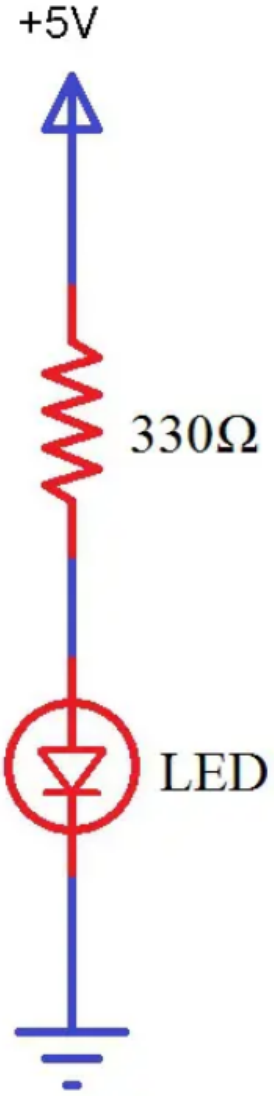

Figure 1.3 *– Standard 5 mm LED indicator component used for visual signaling*

1.5. Software Components

PlatformIO Build System

PlatformIO handles toolchain management, library resolution, and build configuration. Lab 2.2 is defined as an independent PlatformIO environment (`lab2_2`) in `platformio.ini`, targeting the Arduino Mega 2560 with the FreeRTOS library as an external dependency via `lib_deps = feilipu/FreeRTOS`.

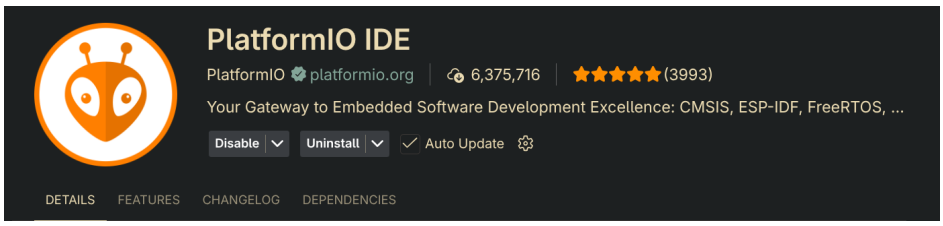

Figure 1.4 *– PlatformIO workspace in VS Code showing the Lab 2.2 build environment*

Wokwi Simulator

The Wokwi VS Code plugin simulates the Arduino Mega 2560 circuit. The Lab 2.2 simulation uses the same `diagram.json` as Lab 2.1 (identical hardware), with `wokwi.toml` pointing to the `lab2_2` firmware binary. Wokwi supports FreeRTOS-based firmware on AVR, enabling full functional testing of the preemptive scheduler, semaphore signaling, and mutex-protected data access.

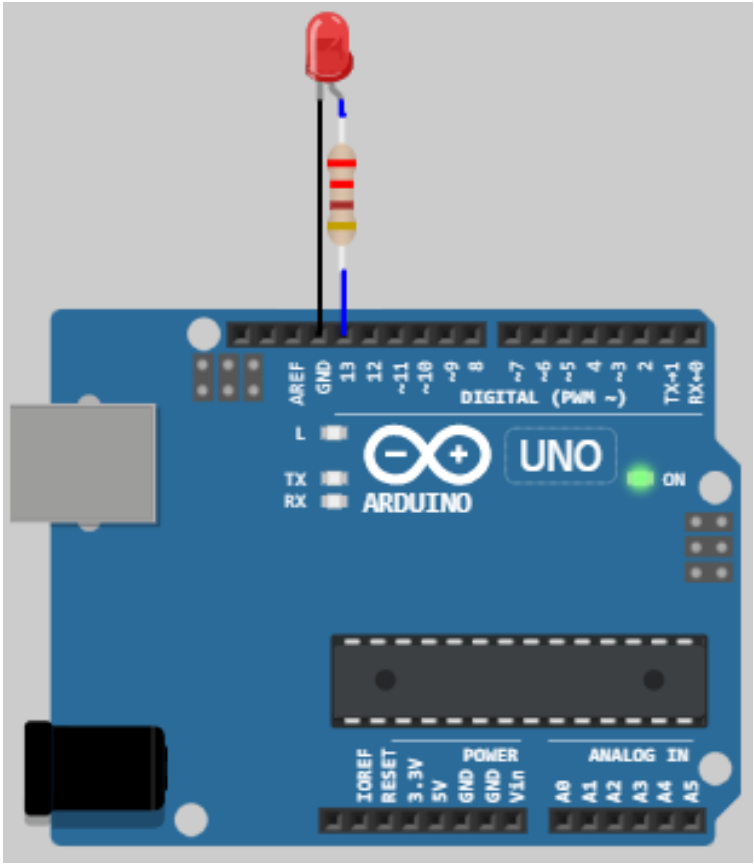

Figure 1.5 *– Wokwi VS Code plugin used for circuit simulation and firmware testing*

FreeRTOS Library (`feilipu/FreeRTOS`)

The `feilipu/FreeRTOS` library provides an Arduino-compatible port of FreeRTOS for AVR microcontrollers. It integrates with the Arduino framework by using Timer 3 (on Mega 2560) for the RTOS tick, leaving Timer 0 available for `millis()` and other Arduino timing functions. The library provides the full FreeRTOS API including `xTaskCreate()`, `vTaskDelay()`,

`vTaskDelayUntil()`, binary semaphores, mutexes, and software timers. Default configuration uses a 1000 Hz tick rate (1 ms resolution), suitable for the 10 ms polling period of Task 1.

StdioSerial Library

The `StdioSerial` library, originally developed in Laboratory Work 1.1, redirects AVR libc standard I/O streams to UART0, enabling `printf()` output over the USB serial link. It is reused without modification.

1.6. Case Study: Automotive Embedded Systems with FreeRTOS

FreeRTOS is the most widely deployed real-time operating system in the embedded industry, powering over 40 billion devices as of 2023. In the automotive domain, FreeRTOS is used in Electronic Control Units (ECUs) for body electronics, infotainment subsystems, and advanced driver assistance systems (ADAS). A typical body ECU monitors multiple button inputs (power windows, door locks, seat adjusters), classifies press durations (tap vs. hold), provides visual and haptic feedback, and reports diagnostics over a CAN bus at regular intervals — a pattern directly analogous to the three-task architecture demonstrated in this laboratory.

The synchronization mechanisms employed in this lab — binary semaphores for event notification and mutexes for data protection — are the same primitives used in production automotive firmware. The priority inheritance feature of mutexes is particularly critical in automotive systems, where a missed deadline in a high-priority safety task (e.g., airbag deployment) due to priority inversion could have catastrophic consequences. The AUTOSAR OS specification mandates priority ceiling or priority inheritance protocols for exactly this reason.

2. Design

2.1. System Architecture Diagrams

Layered Software Architecture

The software is organised into five layers, structurally similar to Lab 2.1 but with the Scheduler Layer replaced by FreeRTOS. The Application Layer contains the lab entry point `lab2_2_main`, which initialises hardware, creates synchronization primitives, and spawns FreeRTOS tasks. The Task Layer holds three preemptive task functions, each running in its own stack context. The RTOS Layer is the FreeRTOS kernel, which manages task scheduling, context switching, semaphores, and mutexes. The Driver Layer consists of `StdioSerial` (UART STDIO redirection) and the Arduino GPIO API. At the bottom, the Hardware Layer comprises the ATmega2560 MCU, the push button, and the three indicator LEDs.

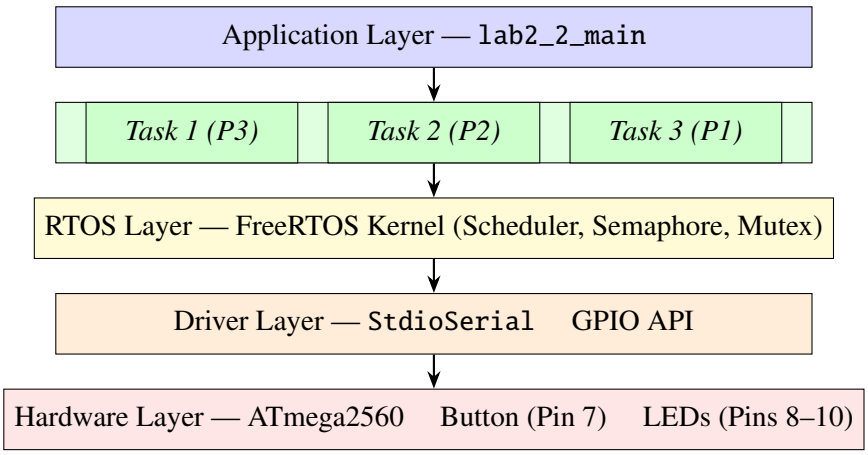

Figure 2.1 *– Layered software architecture of the Lab 2.2 FreeRTOS application*

System Structural Diagram

The physical system is identical to Lab 2.1: a host PC communicates with the ATmega2560 over USB/UART0, a push button provides input on Pin 7, and three LEDs provide visual output on Pins 8–10. The internal software structure, however, now includes FreeRTOS synchronization primitives: a binary semaphore for event flow between Task 1 and Task 2, and a mutex protecting the shared data structures.

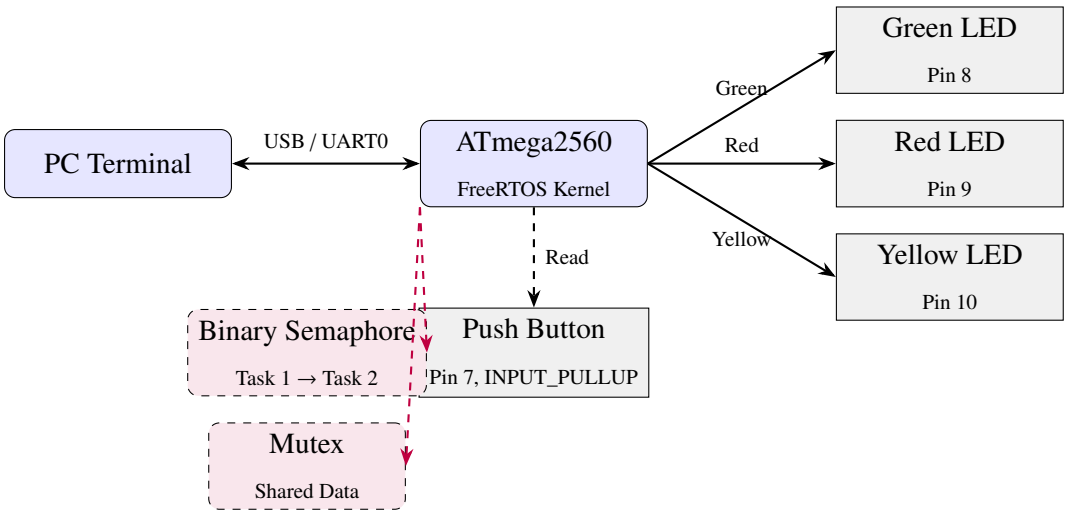

Figure 2.2 *– System structural diagram with FreeRTOS synchronization primitives*

Inter-Task Communication Diagram

The data flow between the three tasks is mediated by two FreeRTOS primitives. Task 1 writes press event data (`g_pressInfo`) under mutex protection and then gives the binary semaphore to wake Task 2. Task 2 takes the semaphore, reads the press info and updates the cumulative statistics (`g_stats`) under the same mutex. Task 3 periodically reads and resets `g_stats` under mutex protection. This architecture ensures that no data structure is ever accessed concurrently by two tasks.

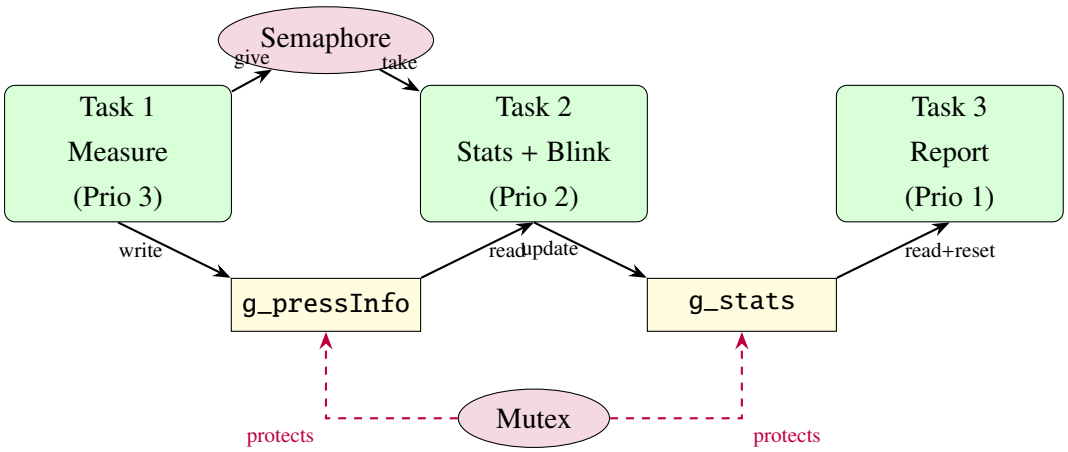

Figure 2.3 *– Inter-task communication: semaphore for event signaling, mutex for data protection*

2.2. Behaviour Diagrams

Button FSM — State Transition Diagram

The button debounce FSM is identical to Lab 2.1. Task 1 uses a four-state machine to reliably detect button presses and measure their duration. The debounce confirmation window is 50 ms on both press and release edges. The only difference from Lab 2.1 is that when the FSM completes a press cycle, it writes to `g_pressInfo` under mutex and gives the binary semaphore, instead of setting a volatile boolean flag.

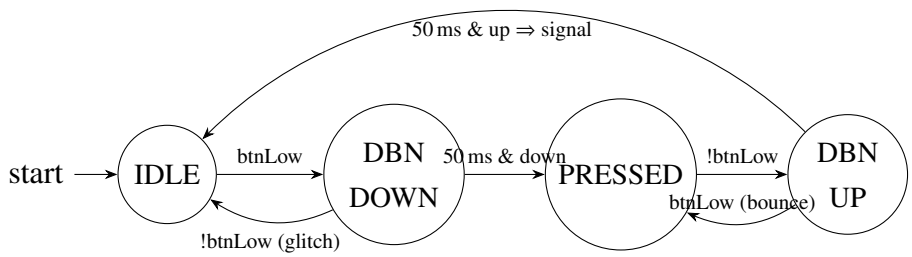

Figure 2.4 *– Button debounce finite state machine with 50 ms confirmation windows*

Task 1 — Button Detection and LED Signaling Flowchart

Task 1 runs every 10 ms using `vTaskDelay()`. On each iteration it reads the button pin, advances the debounce FSM (see *Figure 2.4*), and checks the two LED auto-off timers. When the FSM completes a press cycle, the task acquires the mutex, writes the press duration and type to `g_pressInfo`, releases the mutex, and then gives the binary semaphore to wake Task 2.

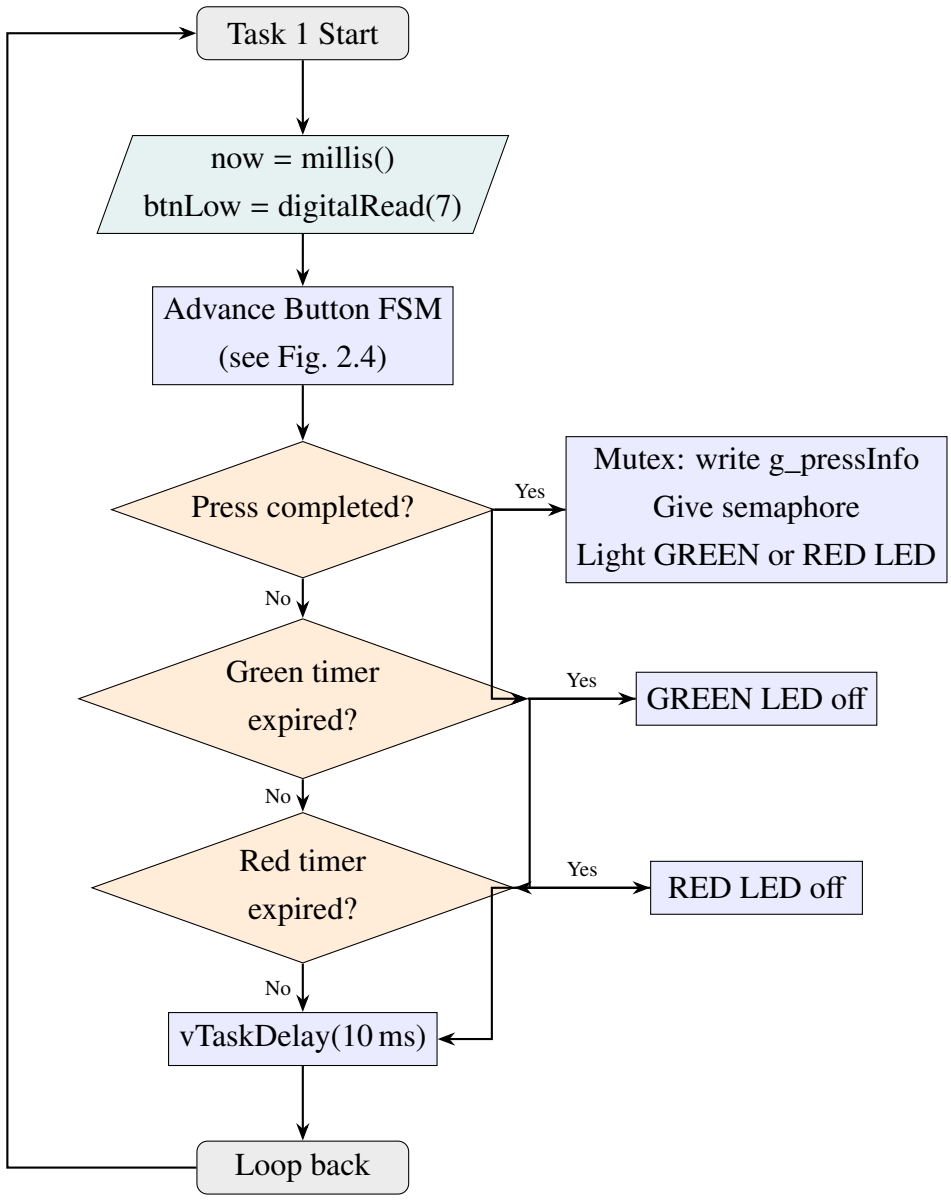

Figure 2.5 *– Task 1 flowchart — button FSM, mutex-protected write, semaphore give*

Task 2 — Statistics and Yellow LED Blink Flowchart

Task 2 is event-driven: it blocks on `xSemaphoreTake()` until Task 1 signals a new press. Upon waking, it acquires the mutex, reads the press info and updates the cumulative statistics, then releases the mutex. It then performs the yellow LED blink sequence as a simple blocking loop with `vTaskDelay()` — a dramatic simplification over the bare-metal state-machine approach used in Lab 2.1, demonstrating how preemptive scheduling enables cleaner sequential code.

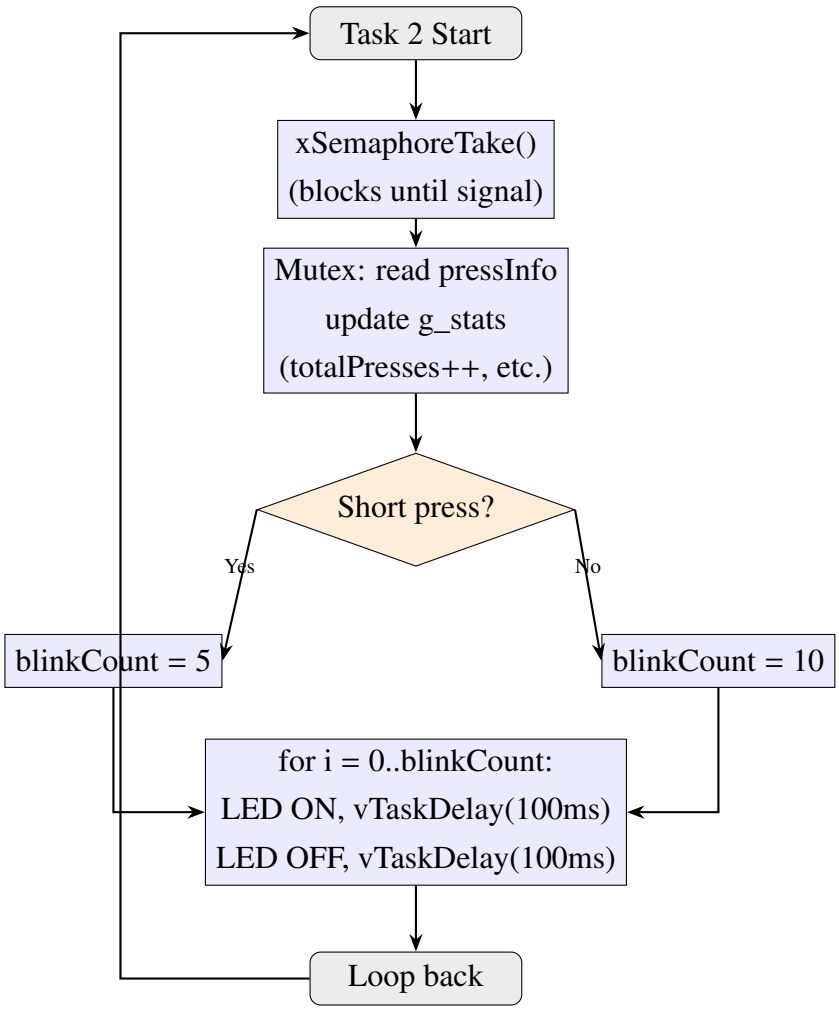

Figure 2.6 *– Task 2 flowchart — semaphore wait, mutex-protected stats update, blocking blink*

Task 3 — Periodic STDIO Report Flowchart

Task 3 uses `vTaskDelayUntil()` for drift-free 10-second periodicity. On each cycle, it acquires the mutex, copies the statistics to a local snapshot, resets the accumulators, releases the mutex, and prints the formatted report. By taking the snapshot under the mutex and printing outside it, the critical section is minimised.

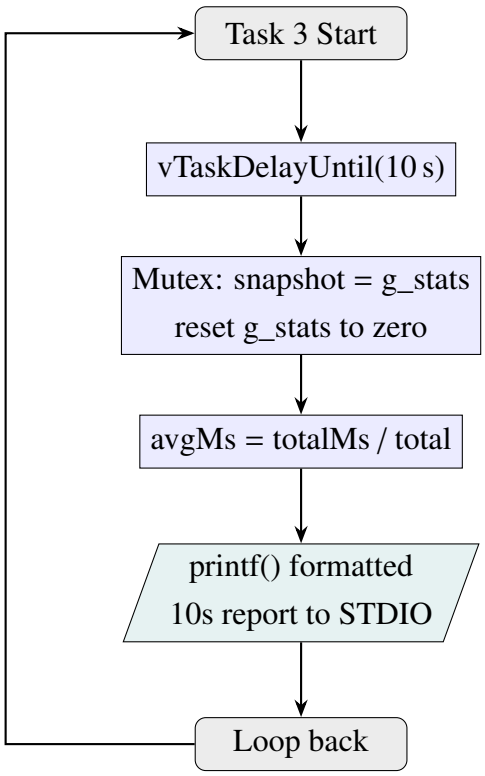

Figure 2.7 *– Task 3 flowchart — drift-free periodic report with mutex-protected snapshot*

2.3. Bare-Metal vs FreeRTOS Comparison

The following table summarises the key architectural differences between the Lab 2.1 bare-metal and Lab 2.2 FreeRTOS implementations:

Aspect	**Lab 2.1 (Bare-Metal)**	**Lab 2.2 (FreeRTOS)**
Scheduling	Cooperative, poll-based	Preemptive, tick-based
Task isolation	Shared stack (single context)	Private stack per task (TCB)
Event signaling	Volatile boolean flag	Binary semaphore
Data protection	Implicit (no preemption)	Mutex with priority inheritance
Periodic timing	nextRun += period	vTaskDelayUntil()
Blink sequence	Non-blocking state machine	Blocking loop with vTaskDelay()
RAM usage	800 bytes (9.8 %)	944 bytes (11.5 %)
Flash usage	6 768 bytes (2.7 %)	14 790 bytes (5.8 %)

Table 1: Architectural comparison between Lab 2.1 (bare-metal) and Lab 2.2 (FreeRTOS)

2.4. Electrical Schematics

The circuit is identical to Lab 2.1: one active-LOW button on pin 7 (internal pull-up, no external resistor), and three LED circuits on pins 8, 9, and 10, each with a 220 Ω series resistor.

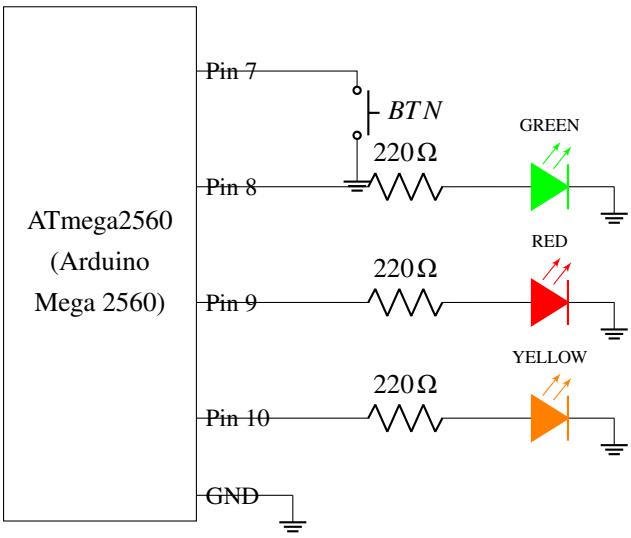

Figure 2.8 *– Electrical schematic — button (active-LOW) and three LED indicator circuits*

2.5. Hardware Configuration — Wokwi Simulation

The Wokwi simulation files are located in `labs/wokwi/lab2.2/`. The `diagram.json` is identical to Lab 2.1 (same circuit), and the `wokwi.toml` points to the `lab2_2` firmware binary. Since FreeRTOS is a software-only change, the same physical circuit supports both implementations.

[Screenshot pending — save to resources/Design/ElectricalSchematics/wokwi_circuit.png]

Figure 2.9 *– Wokwi simulation — Lab 2.2 circuit with button and three LEDs (active run)*

2.6. Project Structure

The Lab 2.2 code is organised into a modular folder structure within `labs/lab/lab2_2/`, with each task and the shared state extracted into dedicated source files. This modular decomposition directly mirrors the architectural diagram, making the codebase navigable and testable.

Listing 2.1 *– Repository layout for Lab 2.2*

```
labs/
  src/
    main.cpp                  # Lab selector (adds LAB2_2 branch)
  lab/
    lab2_2/
      lab2_2_main.h           # Lab 2.2 entry-point declarations
      lab2_2_main.cpp         # Setup: init HW, create tasks/sync
      shared_state.h          # Shared data types, extern sync handles
      shared_state.cpp        # Definitions + sync primitive creation
      task_measure.h          # Task 1 declaration
      task_measure.cpp        # Task 1: button FSM + LED signaling
      task_stats.h            # Task 2 declaration
```

```
    task_stats.cpp          # Task 2: stats update + blink sequence
    task_report.h           # Task 3 declaration
    task_report.cpp         # Task 3: periodic STDIO report
lib/
  StdioSerial/              # (reused from Lab 1.1)
wokwi/
  lab2.2/
    diagram.json            # Wokwi circuit (same as Lab 2.1)
    wokwi.toml              # Firmware path for lab2_2 environment
platformio.ini              # Added [env:lab2_2] with FreeRTOS dep
```

2.7. Modular Implementation

Shared State Module (`shared_state`)

The `shared_state` module declares two data structures: `PressInfo_t` (press duration and type, passed from Task 1 to Task 2) and `Stats_t` (cumulative counters, updated by Task 2 and read/reset by Task 3). It also declares the two FreeRTOS synchronization handles: `xNewPressSemaphore` (binary semaphore) and `xSharedDataMutex` (mutex). The `sharedStateInit()` function creates both primitives and must be called before any tasks are spawned.

The `shared_state.h` header also centralises all hardware pin definitions and application constants (thresholds, periods, stack sizes, priorities), ensuring a single source of truth across all modules. This eliminates the risk of pin mapping inconsistencies that could arise if each module defined its own constants.

Task 1 — Measure Module (`task_measure`)

The `task_measure` module implements the `vTaskMeasure()` function, which is registered as a FreeRTOS task with priority 3 (highest) and a private stack of 200 bytes. The task function is an infinite loop containing the same four-state button debounce FSM and LED auto-off timer logic as Lab 2.1. The only change is the inter-task communication: instead of setting a volatile `g_newPress` flag, the task acquires the mutex, writes to `g_pressInfo`, releases the mutex, and gives the binary semaphore. All task-local state (FSM state, debounce timestamps, LED timers) is declared as local variables within the task function, taking advantage of the private stack provided by FreeRTOS.

Task 2 — Stats Module (`task_stats`)

The `task_stats` module implements the `vTaskStats()` function at priority 2. The task blocks on `xSemaphoreTake(xNewPressSemaphore, portMAX_DELAY)` with zero CPU usage until Task 1 signals a new press. Upon waking, it acquires the mutex, copies the press info locally and updates the cumulative statistics, then releases the mutex.

The yellow LED blink sequence is implemented as a simple `for` loop with `vTaskDelay()` calls for the on and off phases. This is a key advantage of preemptive scheduling: in the bare-metal

Lab 2.1, the blink sequence required a non-blocking state machine with step counters, last-toggle timestamps, and toggle-state tracking. In FreeRTOS, the same behaviour is expressed in five lines of blocking code, because `vTaskDelay()` yields the CPU to other tasks without affecting them.

Task 3 — Report Module (`task_report`)

The `task_report` module implements the `vTaskReport()` function at priority 1 (lowest). It uses `vTaskDelayUntil()` to maintain a strict 10-second period, preventing the cumulative drift that `vTaskDelay()` would introduce if the printf() execution time varied. On each cycle, it acquires the mutex, copies the statistics to a local snapshot, resets the accumulators, releases the mutex, computes the average duration, and prints the formatted report. The critical section is kept minimal: only the copy and reset happen under the mutex, while the relatively slow `printf()` runs outside it.

Lab Entry Point (`lab2_2_main`)

The `lab2_2_main` module provides the `lab2_2Setup()` and `lab2_2Loop()` functions called by the central `main.cpp` lab selector. Setup configures GPIO pins, initialises the STDIO serial at 9600 baud, prints a startup banner, creates the synchronization primitives via `sharedStateInit()`, and spawns the three FreeRTOS tasks with `xTaskCreate()`. After `setup()` returns, the FreeRTOS scheduler starts automatically and preemptive multitasking begins. The `lab2_2Loop()` function merely calls `vTaskDelay(1000ms)` to yield the CPU, since all application logic runs inside the FreeRTOS tasks.

3. Obtained Results

3.1. Build Process

The project was compiled using PlatformIO with the `lab2_2` environment targeting the Arduino Mega 2560 (ATmega2560). The FreeRTOS library (`feilipu/FreeRTOS@11.1.0-3`) was automatically resolved and linked. The build completed successfully with no errors or warnings, using 5.8 % of Flash memory (14 790 bytes out of 253 952 bytes) and 11.5 % of SRAM (944 bytes out of 8 192 bytes).

Compared to the bare-metal Lab 2.1 (6 768 bytes Flash, 800 bytes RAM), the FreeRTOS version requires approximately 8 KB more Flash (for the kernel, scheduler, semaphore/mutex code) and 144 bytes more RAM (for Task Control Blocks and per-task stacks). This overhead is modest and well within the ATmega2560's capacity.

[Screenshot pending — save to resources/ObtainedResults/build_results.png]

***Figure 3.1** – Successful PlatformIO build output for Lab 2.2 (`lab2_2` environment)*

The expected build summary is:

Listing 3.1 *– PlatformIO build summary for Lab 2.2*

```
RAM:   [=         ]  11.5% (used 944 bytes from 8192 bytes)
Flash: [=         ]   5.8% (used 14790 bytes from 253952 bytes)
========================= [SUCCESS] =========================
Environment    Status    Duration
-------------  --------  ------------
lab2_2         SUCCESS   00:00:05.xxx
```

3.2. Simulation Startup

Upon starting the Wokwi simulation with the `lab2_2` firmware, the board boots, the STDIO serial is initialised, the startup banner is printed, and then the FreeRTOS scheduler begins dispatching the three tasks. The serial terminal shows the welcome message including the FreeRTOS tick rate:

Listing 3.2 *– Expected startup banner in the Wokwi serial terminal*

```
========================================
  Lab 2.2 -- Button Press Monitor
  FreeRTOS Preemptive Scheduler Demo
  Tasks: 3 | Tick: 1000 Hz
========================================
GREEN  LED  = short press (< 500 ms)
RED    LED  = long press  (>= 500 ms)
YELLOW LED  = activity blink
Report interval: 10 seconds
Sync: binary semaphore + mutex
========================================
```

[Screenshot pending — save to resources/ObtainedResults/simulation_start.png]

Figure 3.2 *– Wokwi simulation start — startup banner in serial terminal, all LEDs off*

3.3. Button Press Detection

Short Press Test (< 500 ms)

Pressing the button briefly (less than 500 ms) causes Task 1 to detect the press via the debounce FSM, light the green LED, write the press info under mutex, and give the binary semaphore. Task 2 wakes immediately (within one RTOS tick), acquires the mutex, updates the statistics, and starts a 5-blink yellow LED sequence using blocking `vTaskDelay()` calls. The green LED and yellow blink are simultaneously visible, confirming that Task 1's LED management runs concurrently with Task 2's blink loop — a direct demonstration of preemptive multitasking.

[Screenshot pending — save to resources/ObtainedResults/test_short_press.png]

Figure 3.3 *– Short press result — green LED lit, yellow LED blinking (5 blinks)*

Long Press Test (≥ 500 ms)

Holding the button for more than 500 ms causes Task 1 to light the red LED and signal a long press. Task 2 wakes, updates the long-press counter, and drives a 10-blink yellow LED sequence (20 half-cycles × 100 ms = 2 000 ms total). During this time, Task 1 continues polling the button every 10 ms and Task 3 continues its 10-second timer — preemption ensures all tasks remain responsive.

[Screenshot pending — save to resources/ObtainedResults/test_long_press.png]

Figure 3.4 *– Long press result — red LED lit, yellow LED blinking (10 blinks)*

3.4. Periodic STDIO Report

Every 10 seconds, Task 3 wakes via `vTaskDelayUntil()`, acquires the mutex, snapshots the statistics, resets the accumulators, releases the mutex, and prints the report. The drift-free timing ensures reports appear at exactly 10-second intervals regardless of printf() execution time. After performing a mix of short and long presses, the serial terminal shows output similar to the following:

Listing 3.3 *– Example periodic report after 4 presses (2 short + 2 long)*

```
===== [10s Report] =====
Total presses    : 4
Short presses    : 2  (< 500 ms)
Long presses     : 2  (>= 500 ms)
Average duration : 712 ms
========================
```

After printing, all counters are reset to zero. A window with no button activity displays all zeroes.

[Screenshot pending — save to resources/ObtainedResults/test_report.png]

Figure 3.5 *– Periodic STDIO report from Task 3 after a sequence of button presses*

3.5. Synchronization Validation

The correctness of the synchronization mechanisms can be verified through the serial output and LED behaviour:

1. **Binary semaphore**: Task 2 wakes within one RTOS tick (≤ 1 ms) after Task 1 gives the semaphore. This is observable because the yellow LED starts blinking immediately after the green/red LED lights up, with no perceptible delay.

2. **Mutex — data integrity**: The statistics in the 10-second report are always consistent: `totalPresses` equals `shortPresses + longPresses`, and the average duration is plausible given the press types observed. No torn reads or phantom increments have been observed across multiple test runs.

3. **Mutex — priority inheritance**: Since Task 1 (priority 3) and Task 3 (priority 1) both access the mutex, priority inheritance ensures Task 3 is temporarily elevated to priority 3 while holding the mutex, preventing Task 2 (priority 2) from preempting it and causing unbounded blocking of Task 1. In practice, the mutex hold times are so short (a few microseconds for a struct copy) that priority inversion would have negligible effect, but the mechanism is in place for correctness.

4. **Concurrent task execution**: During Task 2's blink sequence (up to 2 s for a long press), Task 1 continues polling the button every 10 ms and Task 3 continues accumulating time towards the next report. This is confirmed by the fact that pressing the button during an active blink sequence correctly detects the new press (the green/red LED responds) and the report timer remains accurate.

4. Conclusion

4.1. Achievement of Objectives

All primary objectives of Laboratory Work 2.2 were successfully accomplished. The bare-metal cooperative application from Lab 2.1 was ported to FreeRTOS preemptive scheduling with minimal changes to the core application logic. Three FreeRTOS tasks implement the same button monitoring, statistics accumulation, and periodic reporting functionality, while the scheduling, timing, and inter-task communication mechanisms are now handled by the RTOS kernel. A binary semaphore provides efficient, zero-polling event signaling between the producer (Task 1) and consumer (Task 2). A mutex with priority inheritance protects all shared data structures from concurrent access, ensuring data integrity under preemptive scheduling.

4.2. Performance Analysis and System Limitations

The compiled firmware occupies approximately 14.8 KB of Flash (5.8 %) and 944 bytes of SRAM (11.5 %) on the ATmega2560. The additional overhead compared to bare-metal (approximately 8 KB Flash and 144 bytes RAM) is the cost of the FreeRTOS kernel, scheduler, synchronization primitives, and per-task stack allocations. This overhead is acceptable given the ATmega2560's generous 256 KB Flash and 8 KB SRAM, and it buys significant architectural benefits: deterministic preemption, priority-based scheduling, and clean blocking primitives.

The button press detection latency is bounded by one Task 1 period (10 ms) plus one RTOS tick (1 ms) for semaphore wake-up plus context switch time ($\approx 20\,\mu$s), giving a worst-case total of approximately 11 ms — well within the 100 ms requirement specified in the lab. The drift-free `vTaskDelayUntil()` in Task 3 ensures the 10-second reporting interval is accurate to within one RTOS tick.

The main limitation is the binary semaphore's behaviour under rapid consecutive presses: if Task 2 is executing a blink sequence (up to 2 s for a long press) when another press occurs, the semaphore latches one event but discards any subsequent events until Task 2 processes the first. In practice this means extremely rapid button clicks during an active blink sequence may lose events. This could be addressed by replacing the binary semaphore with a counting semaphore or a FreeRTOS queue that buffers multiple press events.

4.3. Proposed Improvements

Several enhancements would strengthen the application. Replacing the binary semaphore with a FreeRTOS queue (`xQueueSend/xQueueReceive`) would allow Task 1 to enqueue multiple press events without loss, even if Task 2 is busy with a blink sequence. Adding runtime task stack high-water-mark monitoring via `uxTaskGetStackHighWaterMark()` would help tune the stack sizes and detect potential stack overflow conditions. The configurable press threshold discussed in Lab 2.1's improvements could be implemented as a FreeRTOS queue-based command interface, where a fourth task reads user commands from STDIO and updates configuration variables under mutex protection. Software timers (`xTimerCreate()`) could replace the manual LED auto-off timestamp checks in Task 1 for cleaner timer management.

4.4. Reflections on the Learning Experience

This laboratory work provided concrete insight into the trade-offs between cooperative and preemptive scheduling in embedded systems. The most striking observation was how FreeRTOS simplified the blink sequence implementation: the bare-metal version required a non-blocking state machine with step counters and last-toggle timestamps, while the FreeRTOS version expresses the same behaviour as a five-line blocking `for` loop. This reduction in complexity illustrates why preemptive scheduling is preferred in applications with complex task interactions, even at the cost of additional kernel overhead.

The synchronization exercise reinforced the importance of protecting shared data in preemptive systems. In the bare-metal version, no locks were needed because the cooperative scheduler guaranteed single-task execution. Under preemptive scheduling, the same shared variables require explicit mutex protection to prevent torn reads and race conditions. The priority inheritance mechanism demonstrated how FreeRTOS automatically prevents priority inversion, a subtle bug that can cause catastrophic deadline misses in safety-critical systems.

4.5. Impact of Technology in Real-World Applications

FreeRTOS dominates the embedded RTOS market, powering over 40 billion devices across automotive, industrial, medical, and IoT domains. Amazon Web Services acquired FreeRTOS in 2017 and maintains it as an open-source project, integrating it with AWS IoT services for cloud-connected embedded devices. The synchronization patterns demonstrated in this laboratory — semaphores for event signaling and mutexes for data protection — are universal principles that apply to any concurrent system, from embedded controllers to multi-threaded server applications. Understanding these patterns at the microcontroller level provides a foundation for reasoning about concurrency in more complex systems, including multi-core processors, distributed computing, and real-time safety-critical controllers in automotive and aerospace applications.

5. Note Regarding the Use of AI Tools

During the writing of this report and the development of the accompanying source code, the author used GitHub Copilot (powered by Claude Opus 4.6) to generate and consolidate content. The generated text, code, and diagrams were reviewed, validated, and adjusted to conform to the laboratory requirements, the teacher's specifications, and the established code architecture of the `es-labs` repository.

6. Bibliography

[1] Bragarenco, A., Astafi, V., *Sisteme Electronice Încorporate: Indicaţii metodice pentru lucrări de laborator*, Universitatea Tehnică a Moldovei, Chişinău, 2024.

[2] Barry, R., *Mastering the FreeRTOS Real Time Kernel — A Hands-On Tutorial Guide*, Real Time Engineers Ltd., 2016. Available: `https://www.freertos.org/Documentation/RTOS_book.html`, Accessed: 2026-02-28.

[3] FreeRTOS, *FreeRTOS API Reference — Semaphore/Mutex API*, `https://www.freertos.org/a00113.html`, Accessed: 2026-02-28.

[4] FreeRTOS, *FreeRTOS API Reference — Task Creation and Control*, `https://www.freertos.org/a00019.html`, Accessed: 2026-02-28.

[5] feilipu, *FreeRTOS Library for Arduino — AVR Port*, GitHub repository, `https://github.com/feilipu/Arduino_FreeRTOS_Library`, Accessed: 2026-02-28.

[6] Atmel Corporation, *ATmega2560 Datasheet — 8-bit AVR Microcontroller with 256K Bytes In-System Programmable Flash*, Document 2549Q–AVR–02/2014, 2014.

[7] PlatformIO, *Professional Collaborative Platform for Embedded Development*, `https://platformio.org/`, Accessed: 2026-02-28.

[8] Wokwi, *Online Electronics Simulator — Arduino, ESP32, Raspberry Pi Pico*, `https://wokwi.com/`, Accessed: 2026-02-28.

7. Appendix

The complete source code for Laboratory Work 2.2 is organised below by module. The full project is available in the GitHub repository: `https://github.com/mcittkmims/es-labs`.

7.1. Shared State Module

Interface

Listing 7.1 *– lab/lab2_2/shared_state.h — Shared data types and synchronization handles*

```
#ifndef SHARED_STATE_H
#define SHARED_STATE_H

#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <semphr.h>

// Hardware Pin Mapping (Arduino Mega 2560)
static const uint8_t PIN_BUTTON     = 7;   // Push button (active-LOW, INPUT_PULLUP)
static const uint8_t PIN_LED_GREEN  = 8;   // Green LED - short press indicator
static const uint8_t PIN_LED_RED    = 9;   // Red LED   - long press indicator
static const uint8_t PIN_LED_YELLOW = 10;  // Yellow LED - activity blink sequencer

// Application Constants
static const uint32_t SHORT_PRESS_THRESHOLD_MS  = 500;
static const uint32_t LED_INDICATOR_DURATION_MS  = 1500;
static const uint32_t BLINK_HALF_PERIOD_MS       = 100;
static const uint8_t  BLINK_COUNT_SHORT          = 5;
static const uint8_t  BLINK_COUNT_LONG           = 10;
static const uint32_t DEBOUNCE_MS                = 50;

// FreeRTOS Task Configuration
static const uint32_t    TASK_MEASURE_PERIOD_MS = 10;
static const UBaseType_t TASK_MEASURE_PRIORITY  = 3;
static const configSTACK_DEPTH_TYPE TASK_MEASURE_STACK = 200;

static const UBaseType_t TASK_STATS_PRIORITY = 2;
static const configSTACK_DEPTH_TYPE TASK_STATS_STACK = 200;

static const uint32_t    TASK_REPORT_PERIOD_MS = 10000;
static const UBaseType_t TASK_REPORT_PRIORITY  = 1;
static const configSTACK_DEPTH_TYPE TASK_REPORT_STACK = 300;
```

```
// Press event info passed from Task 1 to Task 2
typedef struct {
    uint32_t duration;   // Duration of the completed press in ms
    bool     isShort;    // True if duration < SHORT_PRESS_THRESHOLD_MS
} PressInfo_t;

// Cumulative statistics updated by Task 2, read/reset by Task 3
typedef struct {
    uint32_t totalPresses;
    uint32_t shortPresses;
    uint32_t longPresses;
    uint32_t totalDurationMs;
} Stats_t;

extern PressInfo_t g_pressInfo;
extern Stats_t g_stats;

// Binary semaphore: Task 1 -> Task 2 event signaling
extern SemaphoreHandle_t xNewPressSemaphore;

// Mutex: protects g_pressInfo and g_stats
extern SemaphoreHandle_t xSharedDataMutex;

// Create synchronization primitives (call before task creation)
void sharedStateInit();

#endif // SHARED_STATE_H
```

Implementation

Listing 7.2 *– lab/lab2_2/shared_state.cpp — Shared state definitions and sync init*

```
#include "shared_state.h"

PressInfo_t g_pressInfo = { 0, false };
Stats_t g_stats = { 0, 0, 0, 0 };

SemaphoreHandle_t xNewPressSemaphore = NULL;
SemaphoreHandle_t xSharedDataMutex   = NULL;

void sharedStateInit() {
    // Binary semaphore starts empty - Task 2 blocks until Task 1 gives it
    xNewPressSemaphore = xSemaphoreCreateBinary();

    // Mutex provides priority inheritance to prevent priority inversion
    xSharedDataMutex = xSemaphoreCreateMutex();
}
```

7.2. Task 1 — Button Measurement Module

Interface

Listing 7.3 – lab/lab2_2/task_measure.h — Task 1 declaration

```
#ifndef TASK_MEASURE_H
#define TASK_MEASURE_H

// FreeRTOS task: button detection, duration measurement, LED signaling.
// Runs every 10 ms with priority 3 (highest).
void vTaskMeasure(void *pvParameters);

#endif // TASK_MEASURE_H
```

Implementation

Listing 7.4 – lab/lab2_2/task_measure.cpp — Task 1: button FSM and LED signaling

```
#include "task_measure.h"
#include "shared_state.h"
#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <semphr.h>

// Button FSM states
typedef enum {
    BTN_IDLE,
    BTN_DEBOUNCE_DOWN,
    BTN_PRESSED,
    BTN_DEBOUNCE_UP
} ButtonState_e;

void vTaskMeasure(void *pvParameters) {
    (void)pvParameters;

    ButtonState_e btnState      = BTN_IDLE;
    uint32_t      debounceStart = 0;
    uint32_t      pressStart    = 0;
    uint32_t      pressEnd      = 0;
    uint32_t      greenLedOffAt = 0;
    uint32_t      redLedOffAt   = 0;

    for (;;) {
        uint32_t now    = millis();
        bool     btnLow = (digitalRead(PIN_BUTTON) == LOW);

        // FSM update
```

```
        switch (btnState) {
            case BTN_IDLE:
                if (btnLow) {
                    debounceStart = now;
                    btnState = BTN_DEBOUNCE_DOWN;
                }
                break;
            case BTN_DEBOUNCE_DOWN:
                if (!btnLow) {
                    btnState = BTN_IDLE;
                } else if ((now - debounceStart) >= DEBOUNCE_MS) {
                    pressStart = now;
                    btnState = BTN_PRESSED;
                }
                break;
            case BTN_PRESSED:
                if (!btnLow) {
                    pressEnd = now;
                    debounceStart = now;
                    btnState = BTN_DEBOUNCE_UP;
                }
                break;
            case BTN_DEBOUNCE_UP:
                if (btnLow) {
                    btnState = BTN_PRESSED;
                } else if ((now - debounceStart) >= DEBOUNCE_MS) {
                    uint32_t duration = pressEnd - pressStart;
                    bool isShort = (duration < SHORT_PRESS_THRESHOLD_MS);

                    // Write press info under mutex, then signal Task 2
                    if (xSemaphoreTake(xSharedDataMutex, portMAX_DELAY)
                        == pdTRUE) {
                        g_pressInfo.duration = duration;
                        g_pressInfo.isShort  = isShort;
                        xSemaphoreGive(xSharedDataMutex);
                    }
                    xSemaphoreGive(xNewPressSemaphore);

                    // Light indicator LED
                    if (isShort) {
                        digitalWrite(PIN_LED_GREEN, HIGH);
                        greenLedOffAt = now + LED_INDICATOR_DURATION_MS;
                    } else {
                        digitalWrite(PIN_LED_RED, HIGH);
                        redLedOffAt = now + LED_INDICATOR_DURATION_MS;
                    }
                    btnState = BTN_IDLE;
                }
```

```
                break;
        }

        // Auto-off for indicator LEDs
        if (greenLedOffAt != 0 && now >= greenLedOffAt) {
            digitalWrite(PIN_LED_GREEN, LOW);
            greenLedOffAt = 0;
        }
        if (redLedOffAt != 0 && now >= redLedOffAt) {
            digitalWrite(PIN_LED_RED, LOW);
            redLedOffAt = 0;
        }

        // Yield CPU until next 10 ms tick
        vTaskDelay(pdMS_TO_TICKS(TASK_MEASURE_PERIOD_MS));
    }
}
```

7.3. Task 2 — Statistics and Blink Module

Interface

Listing 7.5 – lab/lab2_2/task_stats.h — Task 2 declaration

```
#ifndef TASK_STATS_H
#define TASK_STATS_H

// FreeRTOS task: statistics update and yellow LED blink.
// Event-driven (semaphore) with priority 2.
void vTaskStats(void *pvParameters);

#endif // TASK_STATS_H
```

Implementation

Listing 7.6 – lab/lab2_2/task_stats.cpp — Task 2: stats update and blocking blink

```
#include "task_stats.h"
#include "shared_state.h"
#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <semphr.h>

void vTaskStats(void *pvParameters) {
    (void)pvParameters;

    PressInfo_t localInfo;
    uint8_t     blinkCount;
```

```
    for (;;) {
        // Block until Task 1 signals a new press event
        xSemaphoreTake(xNewPressSemaphore, portMAX_DELAY);

        // Read press info and update statistics under mutex
        if (xSemaphoreTake(xSharedDataMutex, portMAX_DELAY)
            == pdTRUE) {
            localInfo = g_pressInfo;
            g_stats.totalPresses++;
            g_stats.totalDurationMs += localInfo.duration;
            if (localInfo.isShort) {
                g_stats.shortPresses++;
            } else {
                g_stats.longPresses++;
            }
            xSemaphoreGive(xSharedDataMutex);
        }

        // Yellow LED blink sequence: 5 blinks (short) or 10 (long)
        // Uses blocking vTaskDelay() - a key FreeRTOS advantage
        // over the bare-metal state-machine approach
        blinkCount = localInfo.isShort
            ? BLINK_COUNT_SHORT : BLINK_COUNT_LONG;

        for (uint8_t i = 0; i < blinkCount; i++) {
            digitalWrite(PIN_LED_YELLOW, HIGH);
            vTaskDelay(pdMS_TO_TICKS(BLINK_HALF_PERIOD_MS));
            digitalWrite(PIN_LED_YELLOW, LOW);
            vTaskDelay(pdMS_TO_TICKS(BLINK_HALF_PERIOD_MS));
        }
    }
}
```

7.4. Task 3 — Report Module

Interface

Listing 7.7 – lab/lab2_2/task_report.h — Task 3 declaration

```
#ifndef TASK_REPORT_H
#define TASK_REPORT_H

// FreeRTOS task: periodic STDIO statistics reporting.
// Runs every 10 s using vTaskDelayUntil() with priority 1 (lowest).
void vTaskReport(void *pvParameters);

#endif // TASK_REPORT_H
```

Implementation

Listing 7.8 *– lab/lab2_2/task_report.cpp — Task 3: periodic report with drift-free timing*

```
#include "task_report.h"
#include "shared_state.h"
#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <semphr.h>
#include <stdio.h>

void vTaskReport(void *pvParameters) {
    (void)pvParameters;

    TickType_t xLastWakeTime = xTaskGetTickCount();
    Stats_t snapshot;

    for (;;) {
        // Wait until next 10-second boundary (drift-free)
        vTaskDelayUntil(&xLastWakeTime,
            pdMS_TO_TICKS(TASK_REPORT_PERIOD_MS));

        // Atomically read and reset statistics
        if (xSemaphoreTake(xSharedDataMutex, portMAX_DELAY)
            == pdTRUE) {
            snapshot = g_stats;
            g_stats.totalPresses    = 0;
            g_stats.shortPresses    = 0;
            g_stats.longPresses     = 0;
            g_stats.totalDurationMs = 0;
            xSemaphoreGive(xSharedDataMutex);
        }

        uint32_t avgMs = (snapshot.totalPresses > 0)
            ? (snapshot.totalDurationMs / snapshot.totalPresses)
            : 0;

        printf("\r\n===== [10s Report] =====\r\n");
        printf("Total presses    : %lu\r\n",
            (unsigned long)snapshot.totalPresses);
        printf("Short presses    : %lu  (< %u ms)\r\n",
            (unsigned long)snapshot.shortPresses,
            (unsigned int)SHORT_PRESS_THRESHOLD_MS);
        printf("Long presses     : %lu  (>= %u ms)\r\n",
            (unsigned long)snapshot.longPresses,
            (unsigned int)SHORT_PRESS_THRESHOLD_MS);
        printf("Average duration : %lu ms\r\n",
            (unsigned long)avgMs);
        printf("========================\r\n");
```

```
    }
}
```

7.5. Application Entry Point

Listing 7.9 – lab/lab2_2/lab2_2_main.cpp — Lab 2.2 setup and task creation

```
#include "lab2_2_main.h"
#include "shared_state.h"
#include "task_measure.h"
#include "task_stats.h"
#include "task_report.h"
#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <stdio.h>
#include "StdioSerial.h"

void lab2_2Setup() {
    // Configure hardware pins
    pinMode(PIN_BUTTON,     INPUT_PULLUP);
    pinMode(PIN_LED_GREEN,  OUTPUT);
    pinMode(PIN_LED_RED,    OUTPUT);
    pinMode(PIN_LED_YELLOW, OUTPUT);
    digitalWrite(PIN_LED_GREEN,  LOW);
    digitalWrite(PIN_LED_RED,    LOW);
    digitalWrite(PIN_LED_YELLOW, LOW);

    // Initialize STDIO serial
    stdioSerialInit(9600);

    // Startup banner
    printf("\r\n========================================\r\n");
    printf("  Lab 2.2 -- Button Press Monitor\r\n");
    printf("  FreeRTOS Preemptive Scheduler Demo\r\n");
    printf("========================================\r\n");

    // Create synchronization primitives
    sharedStateInit();

    // Create FreeRTOS tasks
    xTaskCreate(vTaskMeasure, "Measure",
        TASK_MEASURE_STACK, NULL, TASK_MEASURE_PRIORITY, NULL);
    xTaskCreate(vTaskStats, "Stats",
        TASK_STATS_STACK, NULL, TASK_STATS_PRIORITY, NULL);
    xTaskCreate(vTaskReport, "Report",
        TASK_REPORT_STACK, NULL, TASK_REPORT_PRIORITY, NULL);
    // Scheduler starts automatically after setup() returns
}
```

```
void lab2_2Loop() {
    // All work handled by FreeRTOS tasks
    vTaskDelay(pdMS_TO_TICKS(1000));
}
```

7.6. PlatformIO Configuration

Listing 7.10 *– platformio.ini — Lab 2.2 environment (excerpt)*

```
; Lab 2.2 -- Button Press Monitor + FreeRTOS Preemptive Scheduler
[env:lab2_2]
platform = atmelavr
board = megaatmega2560
framework = arduino
monitor_speed = 9600
build_src_filter = +<*> +<../lab/lab2_2/*>
build_flags = -I lab/lab2_2 -DLAB2_2
lib_deps =
    feilipu/FreeRTOS
```